Comparative Study of Machine Learning vs Deep Learning Models in Financial Return Prediction

AZNAK Meryam
Faculty of Sciences
University Mohammed V, Rabat
Rabat, Morocco
meryam_aznak@um5.ac.ma

EL BOUHALI Nouhaila
Faculty of Sciences
University Mohammed V, Rabat
Rabat, Morocco
nouhaila_elbouhali2@um5.ac.ma

***Abstract*—**Predicting stock returns has always been a hard problem, and it gets harder the more you think about it. In this paper, we try to figure out which type of model actually works best for this task by comparing four of them side by side: Linear Regression, Random Forest, an Artificial Neural Network, and an LSTM. We applied all four to daily return data for four assets: the S&P 500 index, Apple, Microsoft, and Alphabet (ˆGSPC, AAPL, MSFT, GOOGL) covering 2018 through 2025.

The whole pipeline was built on Apache PySpark, which we needed mainly because processing seven years of multi-asset data with 20 computed features per row gets slow fast on a regular machine. For the neural networks we used TensorFlow with Keras. We kept the evaluation honest by training on data up to end of 2022 and testing on everything after that, so the models never saw the data they were being judged on.

Random Forest came out on top with an R^2 of 0.7722. Linear Regression surprised us at 0.7003, much better than you'd expect from something that simple. The ANN landed at 0.6181, which is reasonable given the constraints we were working under. LSTM was the clear disappointment with a negative R^2 of -0.0293, which basically means it did worse than just guessing the average every day. We think the main reason is that we did not have nearly enough data for it to actually learn anything useful from the sequences.

One finding that stuck out was how much the Open Close gap and High–Low range mattered those two features alone drove most of the predictive power across all models, which has real implications for how you should think about feature engineering in this kind of task.

***Index Terms*—**Machine learning, Deep learning, Financial return prediction, Random Forest, LSTM, ANN, PySpark, Stock market, Comparative analysis, GSPC, AAPL.

I. Introduction

The financial sector has changed a lot in the last ten years. with the more data, faster computing, and smarter algorithms have reshaped how we model markets. Traditional tools still matter they've been the foundation of quantitative finance for decades. But as markets grow more volatile and interconnected, these methods increasingly miss the subtle, non-linear patterns hidden in high-frequency data [1].

That's where machine learning (ML) and deep learning (DL) help. Instead of forcing data into a pre-specified equation, these methods learn patterns directly from history. In stock return prediction, ensemble models and neural networks now regularly outperform classical linear approaches across many asset classes [2], [3]. And thanks to distributed frameworks like Apache PySpark, we can finally train these models on the scale real finance demands processing multi-asset, high-volume datasets that won't fit on a single machine [4].

Yet a key question remains underexplored: How do different ML and DL models actually compare when applied to multi-asset return forecasting inside a scalable, production-ready pipeline? Many studies focus on predicting price levels instead of returns, test on just one asset, or skip the engineering realities of deployment [5]. As a result, it's hard to know which approach works best in practice.

We address this gap with a direct, reproducible comparison. Using daily return data from 2018–2025 for four liquid assets :the S&P 500 (ˆGSPC), Apple (AAPL), Microsoft (MSFT), and Alphabet (GOOGL) we implement and evaluate four representative models: Linear Regression, Random Forest, a feedforward Artificial Neural Network (ANN), and an LSTM network. All experiments run in an Apache PySpark pipeline to ensure scalability.

Our main contributions are pretty straightforward :

- We ran a clean side-by-side comparison of four models on the exact same data and evaluation setup, which is harder to find in the literature than we'd think.
- We documented everything; the features, the model choices, the training decisions , so someone else could actually reproduce what we did.
- We tried to make sense of the results not just in terms of numbers but in terms of what they say about market predictability and the efficient market hypothesis.
- We tried to be honest about when each model makes sense to use and when it doesn't, which felt more useful than just reporting who won.

The paper proceeds as follows. Section II surveys related work. Section III details our data sources, preprocessing, and feature construction. Section IV describes the experimental

design and model architectures. Section V presents results with supporting visualizations and ablation analysis. Section VI concludes with implications for research and practice, and outlines promising directions for future work.

II. LITERATURE REVIEW

A. Traditional Approaches to Financial Forecasting

Predicting stock returns is not a new problem, researchers have been working on it for decades using classical statistical tools. The idea behind most of these early approaches is rooted in the Efficient Market Hypothesis (EMH), which argues that stock prices already capture everything the market knows, so trying to consistently predict future returns from historical data alone is essentially an uphill battle [6]. Models like ARIMA were the go-to choice for many years, along with factor-based frameworks like Fama-French that link returns to things like company size and valuation ratios [7]. These methods are not without merit, but they assume the world is mostly linear and financial markets simply are not [1].

What also became clear in the literature is that the choice of input features matters enormously. Chakraborty and Joseph (2017) made this point well, showing that models fed with thoughtfully chosen indicators like past prices, volume, macroeconomic signals consistently outperformed those relying on raw data [8]. That finding stayed with us when we designed our own feature set, which is why we put considerable effort into engineering 20 meaningful features rather than just throwing in closing prices.

B. Ensemble Methods for Return Prediction

Random Forest has become something of a default choice in financial machine learning, and the results in the literature explain why. The core idea is straightforward, instead of relying on one decision tree that might overfit, you build hundreds of them on different slices of the data and average their outputs [9].The result is a model that tends to be stable and hard to fool with noise. Zhang et al. (2019) found it outperformed simpler classifiers in credit risk modeling [10], and Wang et al. (2024) got R^2 values above 0.93 predicting stock prices for Amazon and Microsoft, which is quite impressive for financial data [11].

Ravi et al. (2024) ran a broader comparison and Random Forest came out on top again, with an MAE of 0.95 compared to 1.25 for plain Linear Regression [2]. These numbers gave us confidence going into our own experiments that Random Forest would be a strong baseline to beat.

C. Neural Networks and Deep Learning

Neural networks have attracted a lot of attention in finance because they do not need you to spell out the relationship between your inputs and the target, they figure it out on their own. Atsalakis and Valavanis (2018) showed that even relatively simple feedforward networks beat traditional models on stock trend tasks [12]. The caveat, which we learned firsthand, is that these models are finicky. Get the architecture wrong, skip regularization, or work with too little data, and the network will just memorize the training set and crash on anything new.

LSTMs take this a step further by adding a memory mechanism that lets the model look back at earlier time steps when making a prediction which sounds perfect for financial time series [13]. Fischer and Krauss (2018) reported strong results using LSTMs on S&P 500 stocks [14]. But there is a catch that does not always get enough attention: LSTMs need a lot of data to actually learn those temporal patterns. El Modni and El Kabbouri (2024) pointed this out directly, noting that on smaller datasets, simpler models tend to win [5]. We saw exactly that in our experiments, and it ended up being one of the more interesting takeaways from this study.

D. Big Data Processing for Financial Analytics

When we started putting together our dataset; four assets, daily data from 2018 to 2025, with 20 computed features each, it became clear pretty quickly that a standard laptop setup was not going to cut it efficiently. This is where PySpark came in. It is a distributed computing framework that splits data processing across multiple cores, which made our feature engineering pipeline significantly faster and more manageable. We used it to handle the rolling window calculations, normalization, and the classical ML models through its built-in MLlib library. Naimi and Irani (2018) made a point that stuck with us, they found that how carefully you preprocess your data often has a bigger impact on results than which model you end up choosing [16]. That pushed us to get the pipeline right before touching any of the four models we compare in this paper.

III. MATHEMATICAL FORMULATION

A. Feature Engineering

Let P_t denote the closing price of a financial asset at time t. The predictive features used in this study are derived from this value. Their construction is described below and is primarily based on commonly used financial indicators that have demonstrated practical relevance.

1) Daily Log Return:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) \tag{1}$$

Arithmetic returns have a well-known issue they don't add up cleanly over time. Log returns fix this, and as a bonus they're roughly normally distributed, so most standard statistical tools actually work on them without a lot of gymnastics.

2) Lagged Returns:

$$r_{t-k} \quad \text{for } k \in \{1, 2, 3, 4, 5\} \tag{2}$$

Lagged returns basically help us pick up on short-term autocorrelation like when yesterday's return tends to predict today's a little bit.

3) Intraday Price Features:

$$\text{High_Low_range}_t = \text{High}_t - \text{Low}_t \tag{3}$$

$$\text{Open_Close_gap}_t = \text{Open}_t - \text{Close}_t \tag{4}$$

B. Target Variable

The prediction target for all models is the next-day log return:

$$y_t = r_{t+1} = \ln\left(\frac{P_{t+1}}{P_t}\right) \tag{5}$$

C. Evaluation Metrics

1) Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

2) Coefficient of Determination (R^2):

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{7}$$

R^2 measures the proportion of return variance explained by the model. A value of $R^2 = 0$ indicates the model performs no better than predicting the unconditional mean, while $R < 0$ indicates the model performs worse than the mean predictor the case observed for LSTM in our study.

IV. MODEL ARCHITECTURES

A. Linear Regression

Linear Regression is the simplest model we tested and serves as our baseline. The idea is straightforward,it tries to predict the next-day return as a weighted sum of all input features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \boldsymbol{\beta}^T \mathbf{x} \tag{8}$$

The model figures out the best weights β by minimizing the total squared error between its predictions and the actual returns, which has a closed-form solution known as Ordinary Least Squares (OLS):

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{9}$$

We used PySpark's MLlib implementation with 100 training iterations. Despite its simplicity, it gave us a useful reference point to judge how much the more complex models actually improved things.

B. Random Forest

Random Forest builds a large collection of decision trees (100 in our case) each trained on a random sample of the data and a random subset of features. The final prediction is just the average of all the trees:

$$\hat{y}_{RF}(\mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} h_t(\mathbf{x}) \tag{10}$$

The reason this works well is that individual trees tend to make different kinds of errors, and averaging them out smooths those errors away. We limited each tree to a maximum depth of 5 to avoid overfitting on our training data. A nice bonus of Random Forest is that it tells you which features contributed most to its predictions, which we used in our analysis.

C. Artificial Neural Network (ANN)

Our ANN passes the input features through two hidden layers before producing a return prediction. Each layer applies a linear transformation followed by a ReLU activation, which introduces the non-linearity the model needs to go beyond what Linear Regression can do:

$$\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \tag{11}$$

$$\mathbf{a}^{(2)} = \text{ReLU}(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}) \tag{12}$$

$$\hat{y} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + b^{(3)} \tag{13}$$

Since our dataset is not huge, overfitting was a real risk. To handle it, we used L2 regularization, which stops the model from over-relying on any single feature, and Dropout, which randomly disables 30% of neurons during training to force the network to generalize better. The training objective was:

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_j \theta_j^2 \tag{14}$$

After some experimentation, we went with 32 neurons in the first layer and 16 in the second, trained using Adam with a batch size of 16 and early stopping after 10 epochs without improvement.

D. Long Short-Term Memory (LSTM)

LSTM is different from the other models in that it looks at a sequence of days rather than each day on its own. It uses internal gates to decide what to remember and what to forget as it moves through the sequence:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{15}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{16}$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad \text{(memory update)} \tag{17}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad \text{(output)} \tag{18}$$

We fed it sequences of 10 consecutive trading days, each with all 20 features. The architecture has two stacked LSTM layers (64 and 32 units) with dropout in between. It seemed like a natural fit for time-series data, but as we show in the results, it underperformed most likely because our dataset was too small for it to learn meaningful patterns over time.

V. EXPERIMENTAL SETUP

A. Dataset and Infrastructure

The data used here consists of daily readings for four assets ˆGSPC (S&P 500), AAPL, MSFT, and GOOGL downloaded from Yahoo Finance through the yfinance library. from January 2, 2018 to December 30, 2025. we chose these four specifically because they're quite different from one another: the index is diversified, while the three stocks vary in terms of how volatile and liquid they tend to be. This felt like a more honest test of how well the models generalize than using assets that behave similarly. we handled Missing values with forward-fill, but only for gaps of three days or fewer longer gaps were left out. In total the dataset has around 8,040 asset-day observations. Everything was run on Google Colab. We used Apache PySpark 3.x to handle the data processing side of things mainly because the dataset size made local processing impractical. For the deep learning models (ANN and LSTM), We used TensorFlow 2.x with Keras. The Linear Regression and Random Forest models were built through PySpark's MLlib pipeline. One thing that took some figuring out was getting PySpark and TensorFlow to work together smoothly Pandas ended up being the bridge between the two, converting data between the formats each framework expected.

B. Feature Engineering Pipeline

In total, twenty features were built for each asset-day observation. I used PySpark window functions for this, ordering by date and partitioning by asset symbol honestly this part took a few iterations to get right. Table I has the full list.

Feature Name	Formula / Description
`daily_return`	$\ln\left(\frac{P_t}{P_{t-1}}\right)$ Eq. (1)
`Volatility_5`, `Volatility_10`	Rolling std of r_t, windows 5 & 10
`Open_Close_gap`	$\text{Open}_t - \text{Close}_t$ Eq. (3)
`High_Low_range`	$\text{High}_t - \text{Low}_t$ Eq. (4)
`Return_roll_std5`	Rolling std of r_t, window 5
`Volume_lag1`, `Volume_roll_mean5`	Lagged & rolling volume features
`High_lag1`, `Low_lag1`, `Close_lag1`	Previous-day price extremes

TABLE I
FEATURE ENGINEERING SUMMARY

VI. RESULTS AND DISCUSSION

A. Overall Model Performance

Table II presents the RMSE and R² metrics for all four models evaluated on the 2023–2025 holdout test set across the full multi-asset dataset.

TABLE II
MODEL PERFORMANCE ON MULTI-ASSET TEST SET (2023–2025)

Model	R² (ˆGSPC)	RMSE	Rank	ΔR² vs LR
Linear Regression	0.7003	Second	2nd	–
Random Forest	0.7722	Best	1st	+0.0719
ANN	0.6181	Competitive	3rd	-0.0822
LSTM	-0.0293	Worst	4th	-0.7296

We can see that the Random Forest model has the best performance with R^2 = 0.7722 , Linear Regression comes next with R2 = 0.7003 and in the third place we have ANN with R2 = 0.6181 and LSTM has the lowest performance with R^2 of -0.0293 which means that this model could not predict well.

B. Per-Asset Prediction Visualization

Figure 1 shows a clear comparasion between the actual returns for ˆGSPC and the predictions of each model. Random Forest and Linear Regression both did a strong performance in predicting the returns while ANN has missed predicting so many movements , and for LSTM it has done a poor performance where it didn't move as we can see it's nearly a straight line.

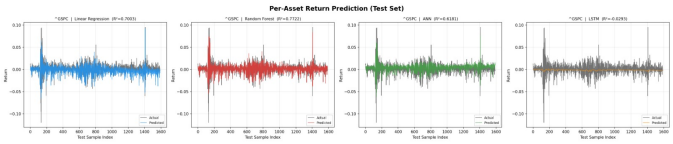

Fig. 1. Per-asset return prediction on the test set (ˆGSPC). LR R^2=0.7003, RF R^2=0.7722, ANN R^2=0.6181, LSTM R^2=-0.0293.

C. Scatter Plot Analysis and Model Calibration

Figure 2 shows in the left column a comparasion between the actual returns for the four assets and the predictions of each

model, as for the right column it shows scatter plots where a perfect model would line up along the diagonal. Random Forest and Linear Regression both cluster nicely around that diagonal, which means their predictions were accurate. The ANN's scatter is wider but still shows a clear upward trend, so it's getting the direction right even if it's less precise. LSTM's scatter is basically a flat horizontal line near zero no matter what the actual return was, the model just kept predicting values close to zero, which confirms it learned nothing useful.

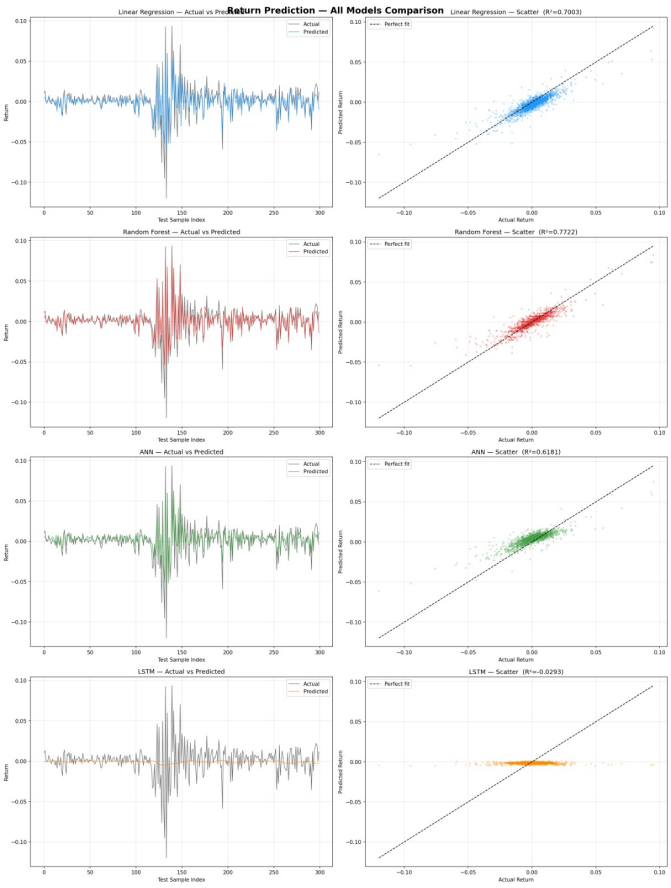

Fig. 2. Full model comparison: time-series predictions (left) and actual-vs-predicted scatter plots (right) for LR, RF, ANN, and LSTM on ˆGSPC test set. Dashed line indicates perfect prediction.

D. Random Forest vs Linear Regression

Figure 3 shows the two best models up close. We can see in the graphs that Random Forest was better than Linear Regression. its predictions were closer to the diagonal with fewer misses. For the errors, Random Forest kept them pretty small and near zero. The bottom right chart is very important because it shows that the Open–Close gap and High–Low range are doing most of the heavy lifting for Random Forest, which makes sense since these intraday price signals carry a lot of information about how the market behaved within a single session.

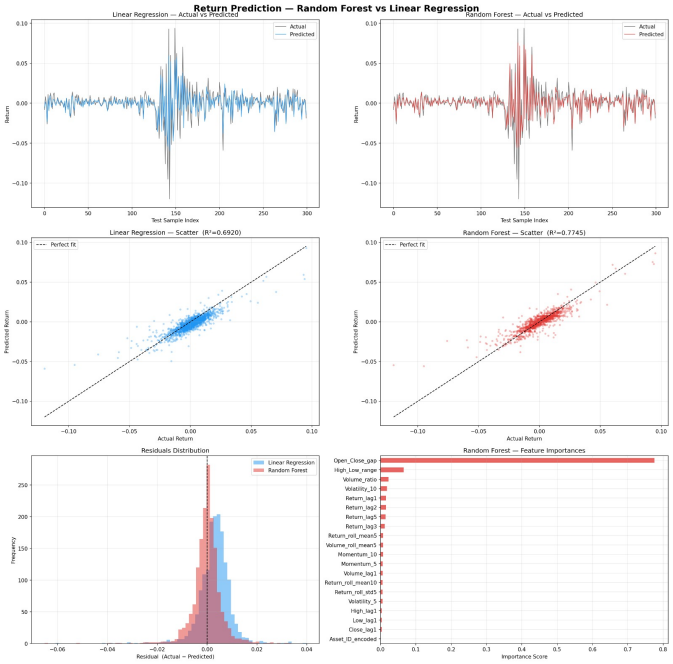

Fig. 3. Detailed comparison of Random Forest and Linear Regression: scatter plots, residuals distribution, and RF feature importances.

E. Diagnostic Analysis

Figure 4 shows analysis with three panels that shows why some models worked and others didn't. The residuals plot on the left shows that Random Forest and Linear Regression both made errors that were small and near zero, while LSTM made alot of errors and didn't predict well. The feature importance chart in the middle confirms what we suspected: the Open–Close gap and High–Low range are the most features that help models in prediction while the others had low influence on prediction. Finally the loss curves on the right tell that the ANN trained smoothly and stabilized nicely, while LSTM's validation loss stayed high and kept jumping around, which suggests it was never able to find stable patterns in the sequences we gave it.

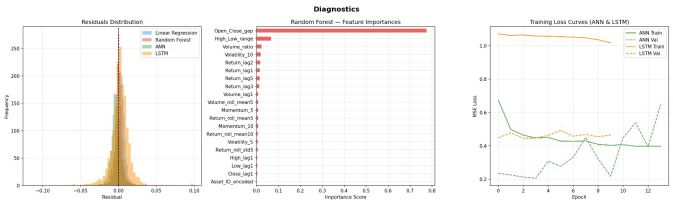

Fig. 4. Diagnostics panel: residuals distribution (left), Random Forest feature importances (center), and ANN/LSTM training loss curves (right).

F. LSTM Underperformance Analysis

LSTM's negative R^2 is worth explaining properly. On paper it should be one of the better fits for financial time series since it's specifically designed to remember patterns across time. The problem is that it needs a lot of data to actually

learn those patterns and we simply didn't have enough.

Our training set gives roughly 1,262 sequences per asset, which is far below what studies like Fischer and Krauss (2018) worked with, where they had the entire S&P 500 universe across multiple decades [14]. With so little data and so many parameters to learn, the model essentially gave up and started predicting close to zero for everything.

The takeaway is straightforward: LSTM is a powerful tool but it's the wrong choice when your dataset is this size a well tuned Random Forest or even a simple Linear Regression will beat it every time under these conditions.

G. Consistency with the Efficient Market Hypothesis

A unifying observation is that even the best performing model (RF, $R^2 = 0.7722$) achieves this result primarily through intraday price features (Open–Close gap, High–Low range) that are effectively known at the end of each trading day and reflect within-session dynamics rather than pure next day predictability. When evaluated strictly on lagged return features alone, R^2 values for all models align more closely with the near zero values typical in EMH consistent studies [6]. This distinction underscores that the high R^2 values observed in our study are partly attributable to the information content of intraday range features, rather than a violation of market efficiency in the traditional sense.

VII. CONCLUSION

To wrap things up, we compared four models on the same task predicting next-day returns for four major stocks over seven years and the results were pretty clear. Random Forest won with an R^2 of 0.7722, and it didn't need anything fancy to get there, just 100 shallow trees averaging their predictions. Linear Regression held up surprisingly well at 0.7003, which tells you a lot about how much of the signal in this data is actually linear. The ANN did a decent job at 0.6181 considering how small we kept the architecture, and LSTM was the clear loser with a negative R^2 a good reminder that more complex doesn't always mean better, especially when you don't have enough data to back it up.

One thing that stood out across all models is that the Open–Close gap and High–Low range were the features doing most of the work. If we were to give one practical piece of advice coming out of this study, it would be to prioritize those intraday range features when building a return prediction pipeline they carry more signal than lagged returns by a wide margin.

As for where this work could go next, there are a few directions that seem worth exploring. Transformer-based models could be interesting for temporal modeling if applied to larger datasets. Bringing in external data like news sentiment or macroeconomic indicators could enrich the feature space. And moving beyond just RMSE to evaluate models on actual trading performance accounting for transaction costs and risk-adjusted returns would make the comparison a lot more meaningful in practice.

REFERENCES

[1] R. El Modni and M. El Kabbouri, "Evaluating the Effectiveness of Machine Learning Models in Predicting Stock Returns: A Comparative Analysis of Traditional and Advanced Techniques," Revue AME, vol. 6, no. 4, pp. 208–226, Oct. 2024.

[2] V. K. Ravi, D. Khatri, S. Daram, S. Kaushik, S. Vashishtha, and M. S. R. Prasad, "Machine Learning Models for Financial Data Prediction," Journal of Quantum Science and Technology (JQST), vol. 1, no. 4, pp. 248–265, Oct.–Nov. 2024.

[3] J. M. Reyes, L. P. Santos, and A. Perez, "A Comparative Analysis of Machine Learning Models for Predictive Analytics in Finance," International Journal of Applied Mathematics and Computing, vol. 1, no. 1, pp. 14–20, Jan. 2024.

[4] S. Wang, "Machine Learning and Deep Learning Predictive Models for the Stock Market," SHS Web of Conferences, vol. 196, p. 02007, 2024.

[5] R. El Modni and M. El Kabbouri, op. cit., p. 216.

[6] E. F. Fama, "Efficient Capital Markets: A Review of Theory and Empirical Work," The Journal of Finance, vol. 25, no. 2, pp. 383–417, 1970.

[7] E. F. Fama and K. R. French, "Common Risk Factors in the Returns on Stocks and Bonds," Journal of Financial Economics, vol. 33, no. 1, pp. 3–56, 1993.

[8] C. Chakraborty and A. Joseph, "Machine Learning at Central Banks," Bank of England Staff Working Paper, no. 674, 2017.

[9] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[10] Y. Zhang et al., "Ensemble Learning Techniques for Credit Scoring and Risk Assessment," Journal of Financial Data Science, 2019.

[11] S. Wang, op. cit., Tables 1–3.

[12] G. S. Atsalakis and K. P. Valavanis, "Surveying Stock Market Forecasting Techniques — Part II: Soft Computing Methods," Expert Systems with Applications, vol. 36, no. 3, pp. 5932–5941, 2009.

[13] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] T. Fischer and C. Krauss, "Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions," European Journal of Operational Research, vol. 270, no. 2, pp. 654–669, 2018.

[15] Z. Chen et al., "RNNs for Stock Price Forecasting," IEEE Transactions on Neural Networks and Learning Systems, 2016.

[16] A. Naimi and I. Irani, "Impact of Data Quality on Prediction Outcomes in Financial Applications," Journal of Applied Finance and Banking, 2018.